CAPSTONE PROJECT

SOUNDVAULT – MUSIC LIBRARY MANAGEMENT SYSTEM

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

in

Btech CSE CORE (Section-A)

to

K.R Mangalam University

By

ADITYA WADHAWAN (2401010023)

Under the supervision of

MS. MANSI KAJAL

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram – 122001, India

January 2026

TABLE OF CONTENTS

S.No.	Contents	Page No.
1.	Abstract	1
2.	Introduction	2
3.	Motivation	3
4.	Literature Review	4
5.	Problem Statement	5
6.	Objectives	6
7.	Tools / Platform Used	7
8.	System Architecture	8–10
9.	Methodology	10–11
10.	References	11–12

ABSTRACT

SoundVault is a web-based Music Library Management System designed to help users efficiently store, search, edit, and manage their personal YouTube music collections in a structured and user-friendly manner. With the rapid growth of digital media consumption, users often face challenges in organising large volumes of music data such as songs and artists. This project provides a centralised solution to overcome these challenges.

The system is developed using the Flask framework in Python, which handles all backend operations, while SQLite serves as a lightweight database to store structured song information. The application allows users to perform complete CRUD operations (Create, Read, Update, Delete), ensuring flexibility and ease of use. The frontend is built with HTML, CSS, and vanilla JavaScript, delivering a clean and responsive interface with an integrated YouTube player for inline playback.

The system improves efficiency by reducing manual effort, enhancing accessibility, and ensuring better organisation of music data. It is particularly suited for personal use and academic demonstration, while also having the potential to be extended into a full-fledged music management platform.

Keywords: Music Management, Web Application, Flask, SQLite, CRUD Operations, YouTube Player, JavaScript, REST API

INTRODUCTION

In the modern digital era, the consumption of music has shifted from physical formats to digital platforms. Users now maintain large collections of songs through online platforms and local references. However, managing such collections without a proper system becomes inefficient and time-consuming.

Traditional file-based storage systems lack proper categorisation, making it difficult to search and retrieve specific data. Users often face issues such as duplicate entries, missing information, and poor organisation. These limitations highlight the need for a structured, database-driven system.

SoundVault addresses these issues by providing a web-based, database-driven solution built on Flask and SQLite. It organises music data into a structured library allowing users to add songs with YouTube links, search by title or artist, edit existing entries, delete records, and play songs directly within the browser via an embedded YouTube player.

The system is designed with future scalability in mind. It can be extended to include features such as user authentication, playlist management, and music recommendations. The modular architecture ensures new functionalities can be integrated without affecting the existing system.

MOTIVATION

The motivation behind developing SoundVault stems from the growing challenges associated with managing music collections manually. Users accumulate large numbers of YouTube links and song references over time, and without a structured system these become difficult to organise, search, or maintain.

Key issues that inspired this project include:

- Difficulty in quickly searching and retrieving specific songs by title or artist
- Lack of a centralised place to store YouTube music links with metadata
- No built-in way to edit or delete saved songs in traditional bookmark systems
- Absence of inline playback without leaving the management interface
- Need for a clean, modern UI that makes library management enjoyable

This project aims to address all these challenges by developing a simple, efficient, and user-friendly system. SoundVault focuses on enhancing user experience through quick search capabilities, inline YouTube playback, and full CRUD control over the music library.

LITERATURE REVIEW

In recent years, various digital management systems have been developed to handle and organise structured data. Database Management Systems (DBMS) play a vital role in storing, retrieving, and managing large volumes of data. DBMS ensures data integrity, reduces redundancy, and provides efficient access mechanisms.

Web frameworks such as Flask have become increasingly popular for developing interactive and scalable applications. Flask is widely used due to its lightweight nature, flexibility, and ease of use. It allows developers to build web applications with minimal configuration, making it especially suitable for small to medium-scale projects.

Several existing music management systems focus primarily on streaming services rather than personal data management. These systems often lack customisation options for users who want to manage their own curated collections. Additionally, most do not provide a modular and scalable design, limiting their usability in small-scale or academic applications.

SoundVault fills this gap by providing a lightweight, self-hosted music library that gives users full CRUD control over their collection, combined with a responsive modern UI and integrated YouTube playback — features not typically found together in academic projects of this scale.

PROBLEM STATEMENT

Managing a personal music collection manually is a time-consuming and inefficient process, especially as the size of the collection grows. Users encounter multiple challenges such as data redundancy, difficulty searching for specific songs, and the absence of a proper organisational structure.

In traditional approaches, music references are stored in unstructured formats such as browser bookmarks or text files, making it difficult to categorise and retrieve information based on parameters like artist or title. Manual handling also does not support update or delete operations efficiently.

Although some digital solutions exist, most are either too complex for personal use, require paid subscriptions, or lack essential features like efficient search, inline playback, and flexible data management. This creates a clear gap between usability and functionality.

There is therefore a strong need for a simple, efficient, and user-friendly system that can systematically organise, store, and manage a personal music collection. The system should minimise redundancy, support full CRUD operations, provide inline playback, and remain accessible for users of all technical levels.

OBJECTIVES

The main objective of this project is to design and develop an efficient and user-friendly Music Library Management System that simplifies the process of managing and organising a personal YouTube music collection. The specific objectives are:

- To design and develop a well-structured web application using Flask and SQLitethat enables organised storage of music records.
- To implement complete CRUD operations — Create, Read, Update, Delete — allowing users to add, view, edit, and remove songs.
- To provide a fast and efficient search functionality enabling users to find songs bytitle or artist.
- To integrate an embedded YouTube player for inline music playback withoutleaving the application.
- To develop a clean, modern, and responsive user interface accessible on bothdesktop and mobile browsers.
- To minimise data redundancy and maintain data consistency through a centraliseddatabase system.
- To expose a well-structured REST API backend that can be extended orconsumed by other clients in the future.
- To build a scalable, modular codebase that supports future enhancements suchas user authentication and playlists.

TOOLS AND TECHNOLOGIES USED

Programming Language: Python

Python is used as the core backend language due to its simplicity, readability, and extensive ecosystem of libraries and frameworks.

Framework: Flask

Flask is a lightweight and flexible micro web framework used for developing the backend of SoundVault. It handles HTTP routing, request processing, and JSON response generation.

Database: SQLite

SQLite is used to store structured data such as songs, artists, and YouTube URLs. It is serverless and lightweight, requiring no additional setup.

Frontend: HTML, CSS, JavaScript

HTML structures the user interface, CSS provides the modern visual styling including animated backgrounds and card layouts, and vanilla JavaScript handles all dynamic interactions and fetch API calls to the backend.

YouTube iFrame API

The YouTube iFrame embed API is used to provide inline video playback within the player section of the UI without navigating away from the application.

Development Tools: VS Code & GitHub

Visual Studio Code is used as the primary code editor. GitHub is used for version control and project management.

SYSTEM ARCHITECTURE

SoundVault follows a three-tier architecture separating the application into distinct layers for better organisation, scalability, and maintainability.

1. Presentation Layer (Frontend)

- Developed using HTML, CSS, and vanilla JavaScript.
- Provides a tab-based interface with Add Song, Search, and Library sections.
- Sends asynchronous fetch requests to the backend.
- Displays results dynamically without page reloads.
- Includes a right-side YouTube player panel for inline playback.

2. Application Layer (Backend)

- Developed using the Flask framework in Python.
- Handles all business logic, routing, and input validation.
- Exposes a REST API with endpoints for Create, Read, Update, and Deleteoperations.
- Returns JSON responses consumed by the frontend JavaScript.

3. Data Layer (Database)

- Uses SQLite as a serverless, file-based relational database.
- Stores song records with fields: id, title, artist, url, is_favorite.
- Handles all insert, update, delete, and select queries via raw SQL throughPython's sqlite3 module.

Request Flow

1. User interacts with the Frontend UI (clicks Add, Search, Edit, or Delete)
2. JavaScript sends an async fetch() request to the Flask backend
3. Flask processes the request and executes the appropriate database query
4. SQLite stores or retrieves the relevant data
5. Flask returns a JSON response to the frontend and JavaScript updates the DOM to reflect the changes

METHODOLOGY

The system follows a structured, iterative development approach ensuring smooth operation, efficient data handling, and seamless frontend-backend integration.

1. Requirement Analysis

Identified the core needs of a personal music library: adding songs, searching, editing, deleting, and playing YouTube videos inline.

2. Database Design

Designed a single songs table in SQLite with fields for id, title, artist, url, and is_favorite. The schema supports all CRUD operations without requiring foreign key relationships.

3. Backend Development

Built RESTful API endpoints using Flask. Each route corresponds to a CRUD operation and returns JSON. Input validation is handled before any database write.

4. Frontend Development

Designed a single-page application using HTML, CSS, and JavaScript. The tab-based navigation switches between Add Song, Search, and Library panels without any page reloads.

5. Integration & Testing

Connected the frontend fetch calls to the backend API endpoints. Tested all CRUD operations — including the edit modal and delete confirmation — in the browser.

6. Version Control

The project is maintained on GitHub with descriptive commit messages tracking each feature addition and bug fix throughout the development cycle.

OUTPUT

The following screenshots demonstrate the working output of SoundVault – the Music Library Management System. The application runs locally on Flask's development server at **http://127.0.0.1:5000** and showcases all implemented CRUD features.

Add Song

The Add Song panel allows users to enter a song title, artist name, and YouTube URL. Clicking the Add to Library button sends a POST request to the /add_song endpoint and saves the record to the SQLite database.

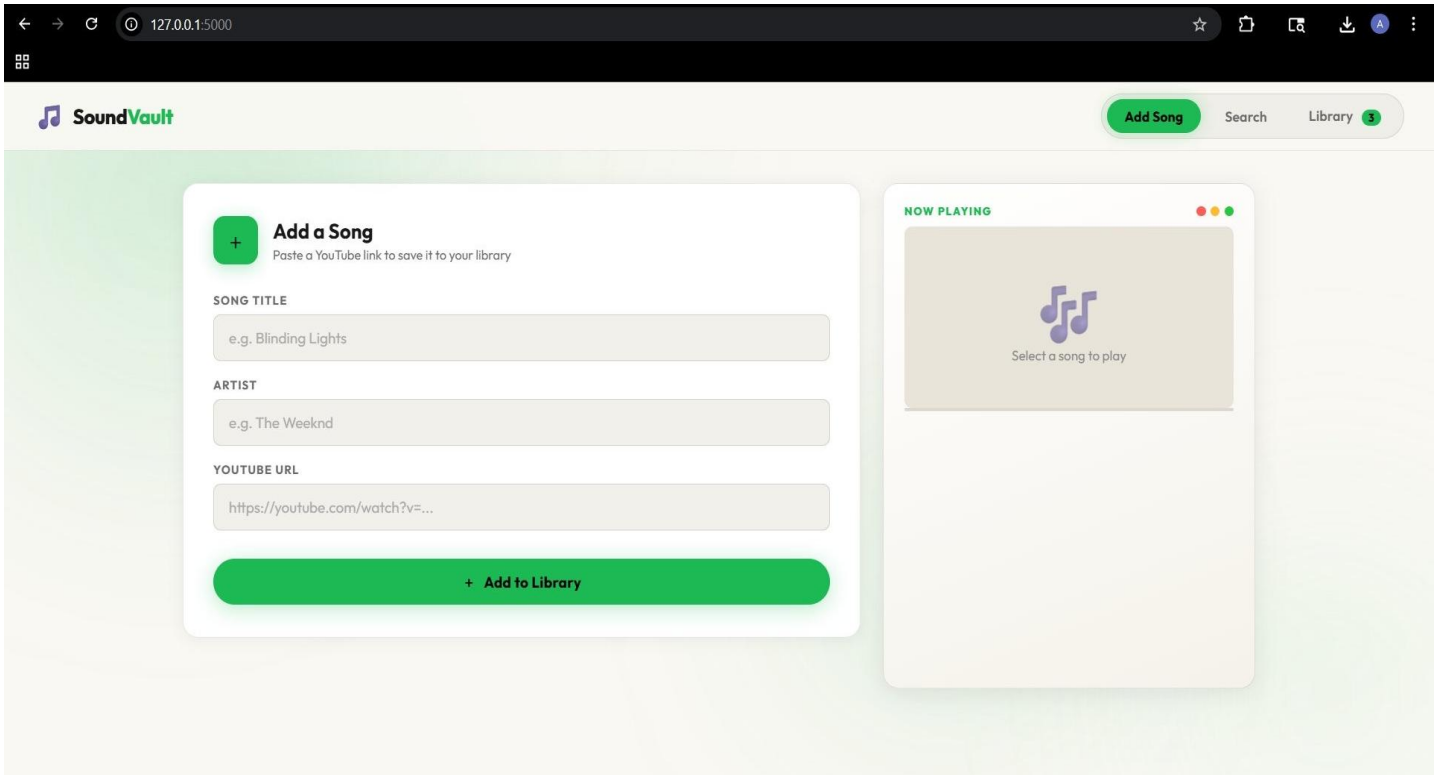
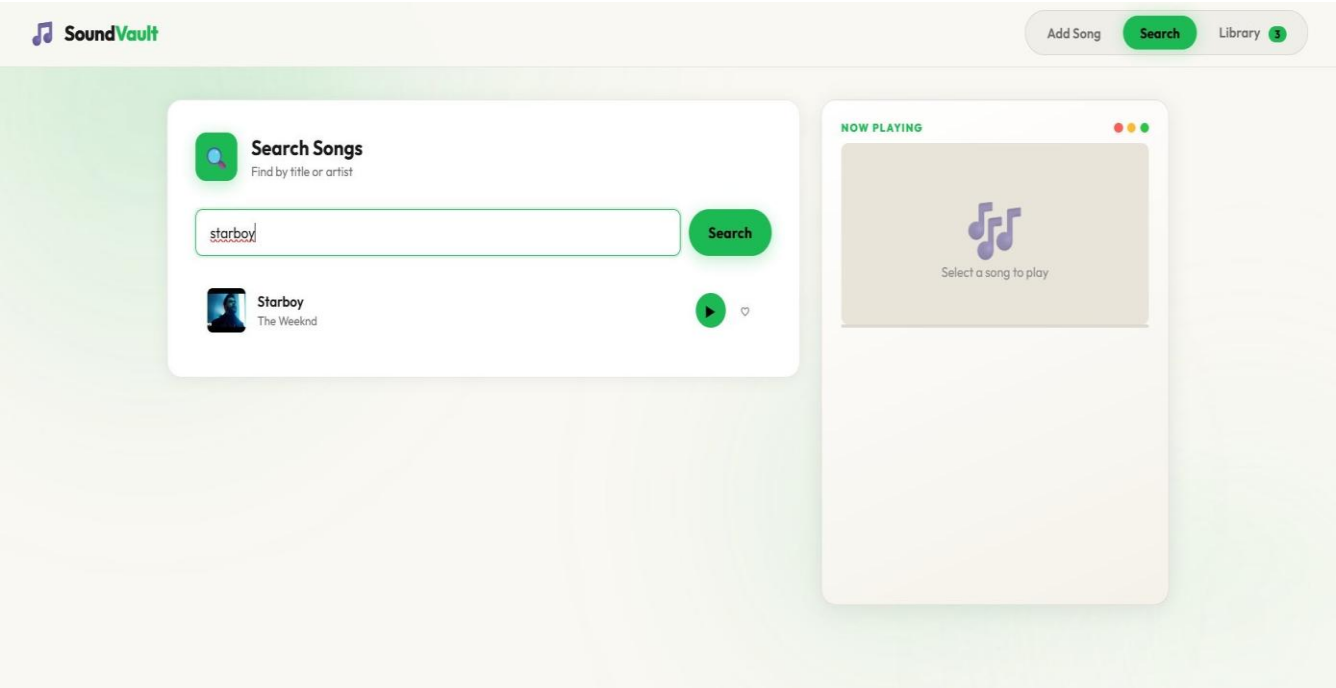


Fig 1: Add Song panel — entering song details with YouTube URL

Search Songs

The Search panel enables users to find songs by title or artist using a LIKE query against the database. Results appear dynamically with a play button and favourite heart icon for each matching song.

Fig 2: Search results for 'starboy' showing matched song with play controls



Your Library – Edit and Delete

The Library panel displays all saved songs ordered by most recently added. Each song card shows a YouTube thumbnail, title, and artist with three action buttons: Play (■), Edit (▬), and Delete (■). The edit and delete functionality represent the Update and Delete operations of the CRUD system.

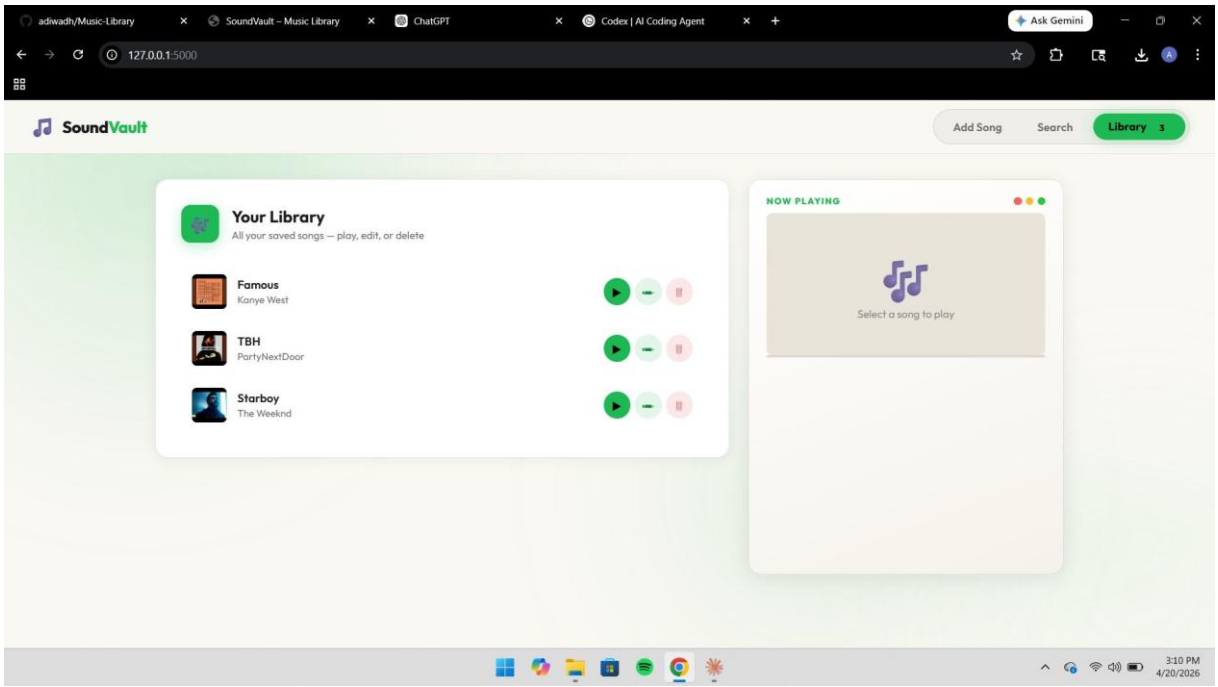


Fig 3: Library view showing 3 songs — Famous, TBH, Starboy — with edit and delete buttons

Inline YouTube Playback

Clicking the Play button on any song loads the corresponding YouTube video into the embedded player on the right side of the interface. The Now Playing section displays the song thumbnail, title, artist, and a progress bar, providing a seamless listening experience without leaving the application.

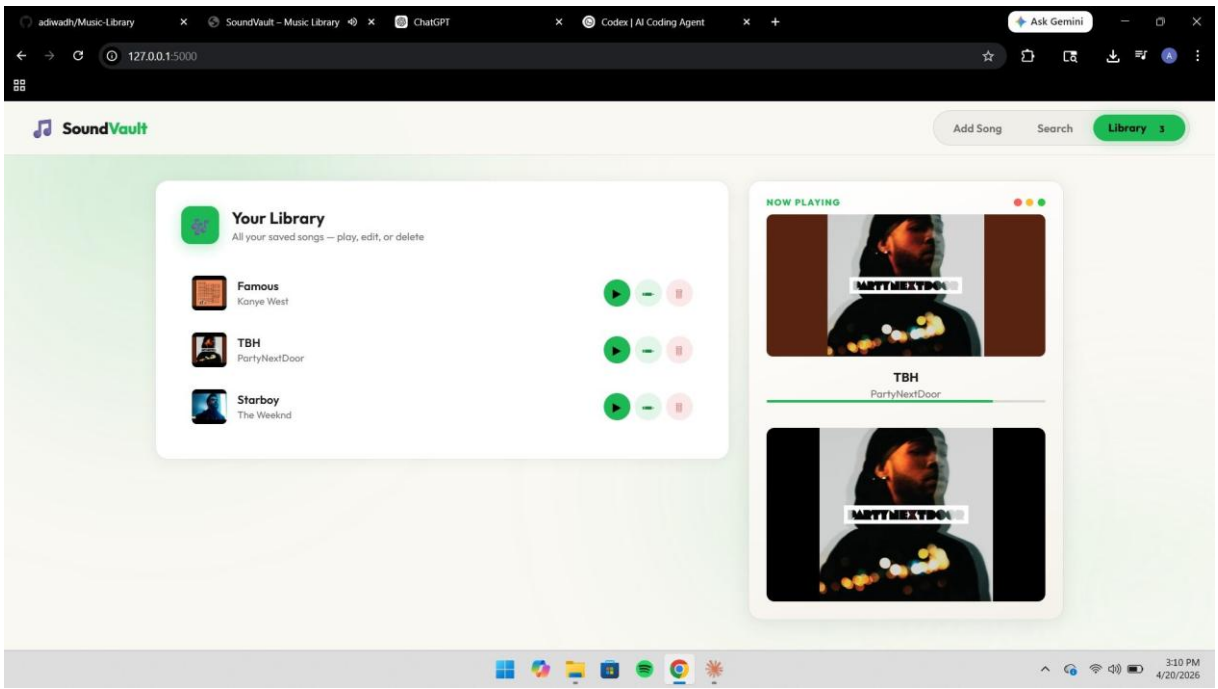


Fig 4: Inline YouTube player showing TBH by PartyNextDoor playing in the Now Playing panel

Feature Summary

Feature	HTTP Method	Endpoint	Status
Add Song	POST	/add_song	✓ Working
View Library	GET	/library	✓ Working
Search Songs	GET	/search?q=	✓ Working
Edit Song	PUT	/update/<id>	✓ Working
Delete Song	DELETE	/delete/<id>	✓ Working
Favourite Song	GET	/favorite/<id>	✓ Working
Inline Playback	—	YouTube iFrame	✓ Working

All features of SoundVault have been successfully implemented and tested. The application demonstrates a complete CRUD cycle backed by a SQLite database and served via a Flask REST API, with a modern, responsive frontend interface.

REFERENCES

1. Flask Official Documentation – https://flask.palletsprojects.com/
2. SQLite Official Documentation – https://www.sqlite.org/docs.html
3. Python Official Documentation – https://docs.python.org/3/
4. MDN Web Docs (HTML, CSS, JavaScript) – https://developer.mozilla.org/
5. W3Schools – HTML, CSS, SQL Tutorials – https://www.w3schools.com/
6. GeeksforGeeks – Flask and Database Tutorials –https://www.geeksforgeeks.org/
7. Stack Overflow – Programming Help and Error Solutions – https://stackoverflow.com/
8. GitHub – Open Source Code Repository – https://github.com/
9. YouTube IFrame Player API – https://developers.google.com/youtube/iframe_api_reference
10. TutorialsPoint – Web Development and Database Concepts – https://www.tutorialspoint.com/
11. Flask Mega-Tutorial by Miguel Grinberg – https://blog.miguelgrinberg.com/
12. SQLite Tutorial – https://www.sqlitetutorial.net/